Claude Code for Academic Researchers

Pre-Workshop Setup Guide

Instructor: Brady Allardice — brady.allardice@upf.edu

Please complete **all eight steps** below before the workshop. The entire process takes 30–50 minutes (plus the large LaTeX download in Step 7, which you can run in the background the day before). If you get stuck on any step for more than 10 minutes, stop and email me at brady.allardice@upf.edu — don't try to debug it yourself. Once you are done, run the verification commands at the end and **send me a screenshot** of the output.

What You Need Before Starting

- A laptop (Mac or Windows) that you can install software on
- An internet connection
- A Claude Pro ($20/month) or Claude Max ($100/month) subscription — sign up at **claude.ai**

The free Claude plan does NOT include Claude Code. You need at least a Pro subscription.

Step 1: Install VS Code

VS Code is the editor we will use throughout the workshop. It is free. If you already have it installed, make sure it is up to date (version 1.98 or newer).

- Go to **https://code.visualstudio.com**
- Download the version for your operating system (Mac or Windows)
- Install it and open it once to confirm it launches

Install the following VS Code extensions (click the Extensions icon in the left sidebar, or press Cmd+Shift+X on Mac / Ctrl+Shift+X on Windows, and search for each):

- **Python** (by Microsoft) — language support for Python
- **R** (by REditorSupport) — language support for R
- **Claude Code** (by Anthropic) — the official Claude Code extension

The Claude Code extension is the primary way you will interact with Claude Code during the workshop. It provides a chat panel inside VS Code where you can issue instructions and see file changes in real time.

Step 2: Install Git and Create a GitHub Account

Git is version control software that tracks changes to your files. We use it throughout the workshop, and Claude Code on Windows requires it to function. GitHub is a platform for hosting Git repositories.

Install Git

Mac: Open the Terminal app (search for "Terminal" in Spotlight) and type:

```
git --version
```

If Git is installed, you will see a version number. If not, your Mac will prompt you to install the Xcode Command Line Tools. Follow the prompt and wait for it to finish.

Windows:

- Go to **https://git-scm.com/downloads/win**
- Download and run the installer
- Use the default settings throughout the installer — do not change anything unless you know what you are doing
- When it asks about the default editor, you can select VS Code if you like

Create a GitHub account (if you don't have one)

- Go to **https://github.com** and sign up for a free account

Connect Git to GitHub

Open a terminal (Mac: Terminal app; Windows: open VS Code and use the integrated terminal via Ctrl+`) and run:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Use the same email address you used to sign up for GitHub.

Step 3: Install Python and Required Packages

The workshop exercises use Python. Even if you normally work in R or Stata, you will need Python installed because Claude Code will write and execute Python scripts during the exercises.

Install Python

- Go to **https://www.python.org/downloads/**
- Download Python 3.11 or 3.12 (either is fine)

Windows — important:

During installation, check the box that says "Add Python to PATH." If you miss this, nothing will work.

Mac: The installer is straightforward. Run it and follow the defaults.

Install required Python packages

After Python is installed, **close and reopen your terminal**, then run:

```
pip install pandas numpy statsmodels matplotlib seaborn scikit-learn openpyxl
```

If you get an error saying `pip` is not found:

Mac:

```
pip3 install pandas numpy statsmodels matplotlib seaborn scikit-learn openpyxl
```

Windows:

```
py -m pip install pandas numpy statsmodels matplotlib seaborn scikit-learn openpyxl
```

If you already have Python installed via Anaconda, that is fine — you likely already have these packages. You can skip this step but please verify by running the verification commands at the end.

Step 4: Install R (if you don't already have it)

If you already have R installed, skip this step. If you are an R user but only have RStudio, you still need R itself installed so that the VS Code R extension and Claude Code can find it.

- Go to **https://cran.r-project.org**
- Download and install R for your operating system
- You do not need RStudio for this workshop, but you can keep it if you have it

Step 5: Install Claude Code (Command Line)

In addition to the VS Code extension you installed in Step 1, we also need the Claude Code command-line tool. The extension and the CLI are two interfaces to the same underlying tool — we will explain the relationship in the workshop. You will primarily use the extension, but understanding the CLI is important.

Mac: Open the Terminal app and run:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Close and reopen your terminal after installation.

Windows: Open PowerShell (search for "PowerShell" in the Start menu) and run:

```
irm https://claude.ai/install.ps1 | iex
```

Close and reopen PowerShell after installation.

Do NOT run PowerShell as Administrator. Claude Code on Windows uses Git Bash internally, which is why Step 2 must be completed first.

Authenticate Claude Code

After installing, open a new terminal and type:

```
claude
```

This will open your browser and ask you to sign in with your Claude account. Follow the prompts, then return to the terminal. You should see the Claude Code welcome screen.

Type `/exit` to close Claude Code for now.

Step 6: Authenticate the VS Code Extension

Open VS Code. Click the Claude Code icon in the left sidebar (the spark icon). It will prompt you to sign in. Follow the browser prompts — this uses the same account as Step 5.

Once authenticated, you should see a chat panel where you can type a message to Claude. Type something simple like "Hello" to confirm it responds.

Step 7: Install a TeX Distribution and the LaTeX Workshop Extension

In Session 3 we will write and compile a LaTeX paper directly inside VS Code, so Claude Code can edit, compile, and version-control your paper alongside your code and data. To do that you need two things: a TeX distribution (the actual compiler) and the **LaTeX Workshop** extension in VS Code (the editor integration).

This is the largest download in the setup (2–4 GB) and can take 30–60 minutes. Do it the day before the workshop, not the morning of.

Install a TeX distribution

Mac — MacTeX:

- Go to **https://www.tug.org/mactex/**
- Download **MacTeX.pkg** (roughly 4 GB)
- Run the installer and follow the defaults

If you want a smaller install, **BasicTeX** (roughly 100 MB) works for this workshop — download it from the same page, then after installation run in Terminal:

```
sudo tlmgr update --self
sudo tlmgr install latexmk collection-fontsrecommended
```

Windows — TeX Live:

- Go to **https://www.tug.org/texlive/windows.html**
- Download **install-tl-windows.exe** and run it
- Use the default "full scheme" installation (roughly 2–4 hours to download; you can leave it running in the background)

If you are short on time or disk space, use the **MiKTeX** distribution instead (**https://miktex.org/download**) — it is smaller and installs missing packages on demand.

Install the LaTeX Workshop VS Code extension

Open VS Code, click the Extensions icon (or press Cmd+Shift+X / Ctrl+Shift+X), search for **LaTeX Workshop** (by James Yu), and install it.

Verify it works

Open a terminal and run:

```
pdflatex --version
```

You should see a version number starting with `pdfTeX 3.x`. If the command is not found, close and reopen your terminal first. On Windows, you may need to log out and back in for the PATH to update.

Step 8: Set Up Zotero

In Session 4 we connect Claude Code to your Zotero library so it can search your references, pull metadata, and generate `.bib` entries for your LaTeX document. To do that you need a Zotero account, the desktop app, a small library to search, and an API key.

Create a Zotero account and install the desktop app

- Go to **https://www.zotero.org/user/register** and create a free account
- Download Zotero from **https://www.zotero.org/download/** (Mac or Windows) and install it
- Open Zotero, sign in via `Edit > Settings > Sync` (Windows) or `Zotero > Settings > Sync` (Mac)

Populate your library

For the Session 4 exercise to be useful, you need at least **10–20 references** in your Zotero library on a topic you care about (your own research, a literature you're starting to read, anything). If your library is empty:

- Install the **Zotero Connector** browser extension from **https://www.zotero.org/download/**
- Use it to grab 10–20 papers from Google Scholar, a journal page, or a syllabus

Generate a Zotero API key

- Go to **https://www.zotero.org/settings/keys** (sign in if prompted)
- Click **Create new private key**
- Give it a name like `Claude Code MCP`
- Check **Allow library access** and **Allow notes access**; leave write permissions **enabled** (we will use the MCP to create annotations and notes)
- Click **Save Key** and **copy the key somewhere safe** — you will not be able to see it again
- On the same page, note your **userID** (the number shown at the top, e.g. `userID: 1234567`)

We will plug the API key and userID into the Zotero MCP configuration during Session 4 — you do not need to install the MCP server in advance.

Verification: Confirm Everything Works

Open a terminal (Mac: Terminal; Windows: PowerShell) and run each of the following commands. **Take a screenshot** of the full output and send it to me.

```
git --version
```

Expected: something like `git version 2.x.x`

```
python --version
```

Expected: `Python 3.11.x` *or* `3.12.x` *(Mac users: try* `python3 --version` *if this fails)*

```
python -c "import pandas; import numpy; import statsmodels; print('All packages OK')"
```

Expected: `All packages OK` *(Mac users: try* `python3` *instead of* `python`*)*

```
claude --version
```

Expected: a version number like `claude-code 1.x.x`

```
R --version
```

Expected: `R version 4.x.x` *(skip this if you are not an R user)*

```
pdflatex --version
```

Expected: `pdfTeX 3.x` *followed by a TeX Live or MiKTeX version line*

Also confirm:

- VS Code opens and the Python, R, Claude Code, and LaTeX Workshop extensions are visible in the Extensions sidebar
- The Claude Code extension panel responds when you type a message
- You have a GitHub account (we will collect usernames on Day 1)
- Zotero desktop opens, your library has at least 10–20 items, and you have saved your API key and userID somewhere safe

Optional: Make VS Code Feel Like RStudio

If you primarily work in R and would rather not give up the RStudio layout (script editor, console, environment, plots, help), VS Code can be configured to look and behave very similarly. **This is entirely optional** — the workshop does not depend on it, and Claude Code works identically either way. Skip this section if you do not use R or are happy with the default VS Code layout.

Install the R packages VS Code needs

Open R (in Terminal/PowerShell type `R`, or open the R GUI) and run:

```
install.packages(c("languageserver", "httpgd", "jsonlite", "rlang"))
```

- **languageserver** powers code completion, hover help, and go-to-definition
- **httpgd** drives the inline plot viewer (the equivalent of RStudio's Plots pane)

Install Radian (a better R console)

Radian is a drop-in replacement for R's default console with syntax highlighting, multi-line editing, and command history. It is what makes the VS Code R terminal feel like RStudio's. Install it via Python (you already have Python from Step 3):

```
pip install -U radian
```

Mac: If `pip` is not found, use `pip3 install -U radian`.

Find where Radian was installed — in the same terminal, run `which radian` (Mac) or `where radian` (Windows) and copy the path.

Install the extra VS Code extension

Open VS Code, go to Extensions (Cmd+Shift+X / Ctrl+Shift+X), and install:

- **R Debugger** (by RDebugger) — step-through debugging for R scripts

The main R extension is already installed from Step 1.

Configure VS Code settings

Open the Settings JSON file: press Cmd+Shift+P (Mac) / Ctrl+Shift+P (Windows), type `Preferences: Open User Settings (JSON)`, and add the following entries inside the outer braces (paste the path from the Radian step into `r.rterm.mac` or `r.rterm.windows`):

```
"r.rterm.mac":  "/usr/local/bin/radian",
"r.rterm.windows":  "C:\\Users\\YOU\\AppData\\Local\\Programs\\Python\\Python312\\Scripts\\radian.exe",
"r.bracketedPaste":  true,
"r.alwaysUseActiveTerminal":  true,
"r.plot.useHttpgd":  true,
"r.sessionWatcher":  true
```

Open the RStudio-style layout

Open any `.R` script in VS Code. With the script focused, press Cmd+Shift+P / Ctrl+Shift+P and run **R: Create R terminal** — a Radian console appears at the bottom. Then open the R Workspace sidebar by clicking the R icon in the activity bar on the left (it appears once you open an `.R` file). You now have:

- Script editor (top) — like RStudio's source pane
- R console (bottom) — like RStudio's console
- R Workspace sidebar (left) — like RStudio's environment + history
- Plot viewer (opens on first `plot()` call via httpgd) — like RStudio's plots pane

Use Cmd+Enter (Mac) / Ctrl+Enter (Windows) to send a line from the script to the console, exactly as in RStudio.

Troubleshooting

"command not found" for any tool: Close your terminal completely and open a new one. If it still fails, the tool was not added to your PATH during installation. Email me with a screenshot of the error.

"command not found" or "zsh: bad pattern: #" after pasting from this PDF: If you copy a multi-line block (anything with a `#` comment, a blank line, or a line break) and paste it into the terminal, the shell will try to run each line as a separate command and choke on the comment or stray characters. **Paste one line at a time**, or strip the `#` comments before pasting. Copying from a PDF can also turn straight quotes (`"`) into curly quotes (`""`), which the shell does not understand — if a command with quotes fails, retype the quotes manually in the terminal.

Claude Code authentication fails: Make sure you have an active Claude Pro or Max subscription at claude.ai. The free plan does not include Claude Code.

Python packages fail to install: Try running the pip command with `--user` at the end, e.g. `pip install pandas --user`. If you are on a managed or institutional machine, you may need to use a virtual environment — email me at brady.allardice@upf.edu and I will help.

Windows: Git not recognized after installing: Make sure you closed and reopened your terminal after installation. If it still fails, the Git installer may not have added Git to your PATH — reinstall and ensure "Git from the command line" is selected.

Windows: Claude Code install fails: Make sure Git for Windows is installed first (Step 2). Claude Code on Windows requires Git Bash internally.

LaTeX: `pdflatex` not found after installing: On Mac, log out and log back in — MacTeX updates PATH via a login item. On Windows, restart your computer after the TeX Live installer finishes.

LaTeX: installer is taking hours: That is normal — TeX Live's full scheme is several gigabytes. Let it run in the background. If it seems stuck, switch to MiKTeX (Windows) or BasicTeX (Mac).

Zotero: lost the API key: You cannot retrieve a saved key. Delete the old one at **https://www.zotero.org/set** and create a new one. Save the new key in a password manager or note immediately.

Anything else: Email me at brady.allardice@upf.edu. Do not spend more than 10 minutes troubleshooting on your own. I would much rather help you before the workshop than lose time during it.